

Лабораторна робота 3

студент 202-Б групи

Павліка Максим Олександрович

Варіант 1

Умова завдання:

Визначити тип: квадратична функція з коефіцієнтами **a**, **b**, **c** ($y=ax^2+bx+c$). Визначити функції:

shift() – зсуву параболи на одиницю вгору;

odd() – визначення парності функції;

roots() – знаходження дійсних коренів рівняння $ax^2+bx+c=0$;

друку координат вершини параболи $y=ax^2+bx+c$;

визначення кількості точок перетину з віссю OX.

Код на C++ (Структура)

```
#include <iostream>

#include <cmath>

#include <cstring>

struct quadraticFunction {
    double a, b, c;
};

void init(quadraticFunction &f) {
    f.a = 0;
    f.b = 0;
    f.c = 0;
}

void input(quadraticFunction &f) {
    std::cin >> f.a >> f.b >> f.c;
```

```
}
```

```
void output(quadraticFunction &f) {  
    std::cout << "f(x) = "  
                << f.a << "x^2 + "  
                << f.b << "x + "  
                << f.c << std::endl;  
}
```

```
char* toPChar(quadraticFunction &f) {  
    char tmp[64];  
  
    snprintf(  
        tmp,  
        sizeof(tmp),  
        "%.2lfx^2 + %.2lfx + %.2lf",  
        f.a,  
        f.b,  
        f.c  
    );  
  
    char* result = new char[strlen(tmp) + 1];  
    strcpy(result, tmp);  
  
    return result;  
}
```

```
void shift(quadraticFunction &f, const double c) {  
    f.c += c;  
}
```

```
inline double discriminant(const quadraticFunction &f) {
```

```

        return f.b * f.b - 4 * f.a * f.c;
    }

bool odd(const quadraticFunction &f) {
    if (-f.b / (2 * f.a) == 0)
        return true;

    return false;
}

int roots(const quadraticFunction &f, double rootsArray[2]) {
    double D = discriminant(f);

    if (D < 0) {
        return 0;
    }

    if (D == 0) {
        rootsArray[0] = -f.b / (2 * f.a);
        rootsArray[1] = rootsArray[0];

        return 1;
    }

    rootsArray[0] = (-f.b + sqrt(D)) / (2 * f.a);
    rootsArray[1] = (-f.b - sqrt(D)) / (2 * f.a);

    return 2;
}

void printVertex(const quadraticFunction &f) {
    double D = discriminant(f);

```

```

        std::cout << "Vertex: ("
                << -f.b / (2 * f.a)
                << ", "
                << -D / (4 * f.a)
                << ")"
                << std::endl;
    }

void printRoots(double rootsArray[2], int count) {
    if (count == 0) {
        std::cout << "Roots: no roots" << std::endl;
    }

    else if (count == 1) {
        std::cout << "Roots: (" << rootsArray[0] << ")" <<
std::endl;
    }

    else {
        std::cout << "Roots: ("
                << rootsArray[0] << ", "
                << rootsArray[1] << ")"
                << std::endl;
    }
}

int main() {
    quadraticFunction func;

    double rootsArray[2];

```

```

init(func);

std::cout << "====INPUT====" << std::endl;
input(func);

std::cout << "====OUTPUT====" << std::endl;

output(func);

int rootsCount = roots(func, rootsArray);

printRoots(rootsArray, rootsCount);

printVertex(func);

if (odd(func))
    std::cout << "Function is even." << std::endl;
else
    std::cout << "Function is odd." << std::endl;

std::cout << "Intersections with OX: "
            << rootsCount
            << std::endl;

std::cout << "====END====" << std::endl;

shift(func, 15);

std::cout << "====OUTPUT====" << std::endl;

```

```

output(func);

rootsCount = roots(func, rootsArray);

printRoots(rootsArray, rootsCount);

printVertex(func);

if (odd(func))
    std::cout << "Function is even." << std::endl;
else
    std::cout << "Function is odd." << std::endl;

std::cout << "Intersections with OX: "
    << rootsCount
    << std::endl;

std::cout << "====END====" << std::endl;

char* converted = toPChar(func);

std::cout << "Conversion: "
    << converted
    << std::endl;

delete[] converted;

return 0;
}

```

Код на C++ (Клас)

```
#include <iostream>
#include <cmath>
#include <cstring>

struct quadraticFunction {
    double a, b, c;
};

class quadraticFunctionClass {
private:
    quadraticFunction func;

public:
    void init();
    void input();
    void output() const;
    char* toPChar() const;

    void setStruct(const quadraticFunction &f);
    quadraticFunction getStruct() const;
};

void quadraticFunctionClass::init() {
    func.a = 0;
    func.b = 0;
    func.c = 0;
}

void quadraticFunctionClass::input() {
    std::cin >> func.a >> func.b >> func.c;
}

void quadraticFunctionClass::output() const {
    std::cout << "f(x) = "
                << func.a << "x^2 + "
                << func.b << "x + "
                << func.c << std::endl;
}

char* quadraticFunctionClass::toPChar() const {
    char tmp[64];

    snprintf(
        tmp,
        sizeof(tmp),
```

```

        "%.2lfx^2 + %.2lfx + %.2lf",
        func.a,
        func.b,
        func.c
    );

    char* result = new char[strlen(tmp) + 1];
    strcpy(result, tmp);

    return result;
}

void quadraticFunctionClass::setStruct(const quadraticFunction &f)
{
    func = f;
}

quadraticFunction quadraticFunctionClass::getStruct() const {
    return func;
}

inline double discriminant(const quadraticFunctionClass &f) {
    quadraticFunction func = f.getStruct();

    return func.b * func.b - 4 * func.a * func.c;
}

void shift(quadraticFunctionClass &f, const double c) {
    quadraticFunction func = f.getStruct();

    func.c += c;

    f.setStruct(func);
}

bool odd(const quadraticFunctionClass &f) {
    quadraticFunction func = f.getStruct();

    if (-func.b / (2 * func.a) == 0)
        return true;

    return false;
}

int roots(const quadraticFunctionClass &f, double rootsArray[2]) {
    quadraticFunction func = f.getStruct();

```

```

double D = discriminant(f);

if (D < 0) {
    return 0;
}

if (D == 0) {
    rootsArray[0] = -func.b / (2 * func.a);
    rootsArray[1] = rootsArray[0];

    return 1;
}

rootsArray[0] = (-func.b + sqrt(D)) / (2 * func.a);
rootsArray[1] = (-func.b - sqrt(D)) / (2 * func.a);

return 2;
}

void printVertex(const quadraticFunctionClass &f) {
    quadraticFunction func = f.getStruct();

    double D = discriminant(f);

    std::cout << "Vertex: ("
                << -func.b / (2 * func.a)
                << ", "
                << -D / (4 * func.a)
                << ")"
                << std::endl;
}

void printRoots(double rootsArray[2], int count) {
    if (count == 0) {
        std::cout << "Roots: no roots" << std::endl;
    }

    else if (count == 1) {
        std::cout << "Roots: (" << rootsArray[0] << ")" <<
std::endl;
    }

    else {
        std::cout << "Roots: ("
                    << rootsArray[0] << ", "
                    << rootsArray[1] << ")"
                    << std::endl;
    }
}

```

```
}
```

```
int main() {
    quadraticFunctionClass func;

    double rootsArray[2];

    func.init();

    std::cout << "====INPUT====" << std::endl;
    func.input();

    std::cout << "====OUTPUT====" << std::endl;

    func.output();

    int rootsCount = roots(func, rootsArray);

    printRoots(rootsArray, rootsCount);

    printVertex(func);

    if (odd(func))
        std::cout << "Function is even." << std::endl;
    else
        std::cout << "Function is odd." << std::endl;

    std::cout << "Intersections with OX: "
                << rootsCount
                << std::endl;

    std::cout << "====END====" << std::endl;

    shift(func, 15);

    std::cout << "====OUTPUT====" << std::endl;

    func.output();

    rootsCount = roots(func, rootsArray);

    printRoots(rootsArray, rootsCount);

    printVertex(func);
```

```

    if (odd(func))
        std::cout << "Function is even." << std::endl;
    else
        std::cout << "Function is odd." << std::endl;

    std::cout << "Intersections with OX: "
                << rootsCount
                << std::endl;

    std::cout << "====END====" << std::endl;

    char* converted = func.toPChar();

    std::cout << "Conversion: "
                << converted
                << std::endl;

    delete[] converted;

    return 0;
}

```

Скріншоти виконання:

```

====INPUT====
1 1 -6

====OUTPUT====
f(x) = 1x^2 + 1x + -6
Roots: (2, -3)
Vertex: (-0.5, -6.25)
Function is odd.
Intersections with OX: 2
====END====

====OUTPUT====
f(x) = 1x^2 + 1x + 9
Roots: no roots
Vertex: (-0.5, 8.75)
Function is odd.
Intersections with OX: 0
====END====
Conversion: 1.00x^2 + 1.00x + 9.00

```